

Companion Technical Report associated to the paper “Modeling, Formalizing, and Animating Environment-Aware BPMN Collaborations”

Abstract. In this technical report, we provide the full account of the formal semantics of the *environment-aware BPMN collaboration* models introduced in the paper “Animating Environment-Aware BPMN Collaborations: From Formal Semantics to Tool Support”, which is under submission.

1 Formal syntax

In this section, we provide the formalization of the syntax of *environment-aware BPMN collaboration* models. Our syntax extends the one proposed in [1] with new BPMN elements, specifically devised for dealing with the interplay between the collaboration processes and the environment. Most of all, our formalization consider in addition a richer model of the environment. To keep the formalization manageable and understandable, we focus on elements that are strictly needed to define meaningful collaborations, the environment where the collaboration happens, and the relations between them.

The formalization resorts to a textual representation that uses a Backus-Naur Form (BNF) syntax for representing the new BPMN elements and the extension of the environment. The motivation for using here a textual representation rather than the usual graphical notation is that the former is more manageable for writing operational semantic rules than the latter. In addition, the textual notation makes explicit those technical details of collaboration models that are not part of the graphical representation (e.g., message payloads, assignments, guard conditions), but are part of the low-level XML characterization of the model. This information is needed to properly define the execution semantics of the environment-aware BPMN collaboration models.

Figure 1 reports the BNF syntax defining the textual notation describing the structure of environment-aware BPMN collaboration models. Specifically, the upper table reports the grammar productions defining the syntax of collaboration models together with the notation of the related generic elements of the syntactic categories. The symbol $*$ stands for (possibly empty) sequences, and $+$ for non-empty sequences. As usual for BPMN models, we assume that message flows, sequence flows, and task names are unique in the model; similarly, we also assume that handshake flows are unique. Notably, even if this syntax would allow writing collaboration models that cannot be expressed in BPMN, we will consider only those terms of the syntax that can be derived from BPMN models. The bottom table reports the definition of the environment where a collaborations is enacted, together with the related notation.

Intuitively, a environment-aware BPMN collaboration is the union of the terms C , indicating the structure of the collaboration, and ENV , indicating the environmental

$C ::=$	$\text{pool}(p, P) \mid C \parallel C'$	(Collaboration Structures) (pool, pool composition)
$P ::=$	$\text{start}(f, f') \mid \text{end}(f)$ $\mid \text{snd}(f, m, \text{exp}, f')$ $\mid \text{rcv}(f, m, \text{do.i}, f')$ $\mid \text{andSplit}(f, F) \mid \text{andJoin}(F, f)$ $\mid \text{xorSplit}(f, (\text{exp}, f)^+) \mid \text{xorJoin}(F, f)$ $\mid \text{eventBased}(f, (m, \text{do.i}, f')^+)$ $\mid T \mid P \mid P'$	(Process Structures) (start event, end event) (msg sending intermediate event) (msg receiving intermediate event) (AND split/join gateway) (XOR split/join gateway) (event-based gateway) (tasks and process composition)
$T ::=$	$\text{task}(f, n, \text{exp}, A, d, f')$ $\mid \text{taskM}(f, n, \text{exp}, A, \text{exp}', f')$ $\mid \text{taskB}(f, h, f')$ $\mid \text{taskU}(f, h, f')$	(basic task) (movement task) (binding task) (unbinding task)
$A ::=$	$\text{assign}(\text{exp}, v)^*$	(Assignments)
(Notation)		
Pool name: p		Data object name: do
Sequence flow: f		Physical place name: pp
Sequence flow set: F		Logical place name: lp
Message flow: m		Edge name: e
Handshake flow: h		Field name: i
Expression: exp		Attribute name: a
Task name: n		Variable (i.e., do.i , pp.a , lp.a , $e.a$): v

$ENV ::=$	(PL, LL)	(Environment Model)
$PL ::=$	$(\mathbb{PP}, \mathbb{E}, E, \mathbb{A}_{pp}, \mathbb{A}_e)$ with $E \subseteq \mathbb{PP} \times \mathbb{E} \times \mathbb{PP}$	(Physical Layer)
$LL ::=$	(V^*, LP) with $LP : \mathbb{LP} \rightarrow \mathbb{EXP}$	(Logical Layer)
$V ::=$	$(\mathbb{LP}, \mathbb{A}_{lp}, A_{lp})$ with $A_{lp} : \mathbb{A}_{lp} \rightarrow (\mathbb{AG} \times \mathbb{DAG})$	(View)

(Notation)		
Physical place names set: $\mathbb{PP}$		Logical place names set: $\mathbb{LP}$
Edge names set: $\mathbb{E}$		Logical place attributes: $\mathbb{A}_{lp}$
Edges: E		Attribute (dis)aggregation: A_{lp}
Physical place attributes: $\mathbb{A}_{pp}$		Expressions set: $\mathbb{EXP}$
Edge attributes: $\mathbb{A}_e$		Aggregation functions set: $\mathbb{AG}$
Logical place definition function: LP		Disaggregation functions set: $\mathbb{DAG}$

Fig. 1: BNF syntax of environment-aware BPMN collaboration models.

model. A collaboration is rendered in the presented syntax as a collection of pools coupled with an environmental model divided into a physical layer and a logical layer. Formally, a collaboration C is a composition by means of the operator $\parallel$, of pool elements $\text{pool}(p, P)$ uniquely identified by a pool name p and enveloping process structures of the form P . Similarly, a process P is a composition of process elements (denoted by the sans serif font) by means of the operator $|$. We identify collaborations (resp. processes) up to commutativity and associativity of pool (resp. process element) collection. Thus, e.g., $C_1 \parallel C_2$ and $C_2 \parallel C_1$ are identified as the same collaboration due to the commutativity property, while $P_1 | (P_2 | P_3)$ and $(P_1 | P_2) | P_3$ are identified as the same process due to the associativity property. To support a compositional approach, in the textual notation each sequence/message flow of the graphical notation is split into two parts: the part outgoing from the source element and the part incoming into the target element. The two parts are correlated by a unique sequence/message flow name.

More specifically, the elements of the notation are as follows.

- $\text{pool}(p, P)$ denotes a pool identified by a name p , enveloping a process structure P .
- $\text{start}(f, f')$ denotes a start event that contains an incoming (spurious) sequence flow f , named enabling flow, used to activate the start event, and the outgoing sequence flow f' .
- $\text{end}(f)$ denotes the end event and f is the incoming sequence flow.
- $\text{snd}(f, m, \text{exp}, f')$ denotes the message intermediate sending event that is characterized by an incoming sequence flow f , a message m , an expression exp , whose evaluation will return the value to be sent, and an outgoing sequence flow f' .
- $\text{rcv}(f, m, \text{do.i}, f')$ denotes the message intermediate receiving event that is characterized by an incoming sequence flow f , a message m , a data object field do.i , and an outgoing sequence flow f' .
- $\text{andSplit}(f, F)$ denotes an AND split gateway characterized by an incoming sequence flow f and a set of outgoing sequence flow F .
- $\text{andJoin}(F, f)$ denotes an AND join gateway characterized by a set of incoming sequence flow F and an outgoing sequence flow f .
- $\text{xorSplit}(f, (\text{exp}, f)^+)$ denotes an XOR split gateway characterized by an incoming sequence flow f and a list of outgoing sequence flows, each one specifying a triggering condition exp .
- $\text{xorJoin}(F, f)$ denotes XOR join gateways characterized by a set of incoming sequence flow F and an outgoing sequence flow f .
- $\text{eventBased}(f, (m, \text{do.i})^+)$ denotes an event-based gateway which is similar to the XOR-split gateway, but its outgoing sequence flow activation depends on taking place of message receiving events.
- $\text{task}(f, n, \text{exp}, A, d, f')$ denotes a basic task, which has a name n , a guard exp that is a conditional expression predicating on attributes, a list of assignments A , and a duration d that represents the amount of time it takes to execute the task. The task has also an incoming and an outgoing sequence flow. Note that, for the sake of readability, when the guard and/or the assignments are not used, they are omitted from the syntactic definition of the element.
- $\text{taskM}(f, n, \text{exp}, A, \text{exp}', f')$ denotes a movement task, which is a type of task that may involve a movement in the environment; it is similar to the basic task but it has

an additional expression exp' that represents the destination to reach. In addition, a movement task does not have the term d because it is assumed that its duration is given by the time to reach the destination.

- $\text{taskB}(f, h, f')$ denotes a binding task, which specifies a handshake flow h that connects it to the binding task of another participant (which has the same h).
- $\text{taskU}(f, h, f')$ denotes an unbinding task, which specifies a handshake flow h that connects it to the unbinding task of another participant (which has the same h).
- $\text{assign}(\text{exp}, v)$ denotes an assignment that assigns the value resulting from the evaluation of the expression exp to a variable, which can be a data object field do.i , an attribute of a physical place pp.a , an attribute of a logical place lp.a , or an attribute of an edge e.a .

Let us focus now on the syntax of the environment model. An environment ENV is a tuple (PL, LL) where PL represents the physical layer while LL represents the logical layer.

The physical layer PL consists of a place graphs, i.e. a graph whose nodes represent physical places (whose names are in the set $\mathbb{PP}$) and directed arcs represent edges E (whose names are in the set $\mathbb{E}$) connecting places. Both physical places and edges are equipped with contextual information represented in terms of attributes (whose names belong to the sets $\mathbb{A}_{pp}$ and $\mathbb{A}_e$, respectively).

The logical layer LL consists of a set of views and a function $LP : \mathbb{LP} \rightarrow \mathbb{EXP}$ that specifies which physical places form each logical place. Notably, the physical places forming a logical place are not statically listed in the model, because the constituents of a logical place may change dynamically. Thus, the LP function maps each logical place to a boolean expression on physical place attributes, whose evaluation on a physical place determines if it belongs (at the time of such an evaluation) to the corresponding logical place. Formally, the set of physical places forming a given logical place $\text{lp} \in \mathbb{LP}$ is defined as follows: $\mathcal{P}(\text{lp}) = \{\text{pp} \in \mathbb{PP} \mid [LP(\text{lp})]_{\text{pp}}\}$, where function $[\cdot]_{\text{pp}}$ returns an expression resulting from the instantiation of the input expression with the physical place name pp , i.e., $[\text{exp}]_{\text{pp}}$ replaces each attribute a in exp with pp.a .

Finally, a view V groups a set of logical places $\mathbb{LP}$, defining how the values of their attributes (in the set $\mathbb{A}_{lp}$) result from the aggregation of the values of physical place attributes and, vice versa, propagate from the logical to the physical layer. Specifically, the handling of the logical place attributes of a view is defined by the function A_{lp} that maps each logical attribute to an aggregation and a disaggregation function. The aggregating function of a logical attribute is called during the evaluation of an expression when the logical attribute is encountered; the disaggregating function, instead, is called when an assignment, having the logical attribute as target, is processed. In practice, these functions provides a transparent mechanism to access and update the value of a logical attribute (like a C# property). This allows the modeller to simply deal with logical attributes as standard (i.e., physical) attributes in expressions and assignments, thus relieving him/her from the duty of explicitly handling aggregation and disaggregation of values.

2 Formal Semantics

To describe the semantics of environment-aware BPMN collaboration, we enrich the structural part of the model with the execution state, given by: the marking of sequence flows with tokens [2, p. 27]; the value of data object fields, physical place attributes and edge attributes; the binding among participants; the status of tasks; the position of the participants in the place graph; the status of timers; and the payloads of the exchanged messages. These stateful descriptions are called *configurations*; we have three kinds of configurations: a *process configuration* has the form $\langle P, ENV, p, \sigma \rangle$, a *collaboration configuration* has the form $\langle C, ENV, \sigma \rangle$, and an *open-world environment configuration* has the form $[C, ENV, \sigma]$, where σ is an execution state. The last configuration differs from the previous one because it allows the environment to evolve freely and independently from the collaboration execution. The environment, indeed, can change based on external factors, such as the traffic on a road.

As mentioned before, the execution state carries many information concerning the execution of the environment-aware BPMN collaboration. In fact, the state σ consists of the following list of *state functions*:

- $\sigma_f : \mathbb{F} \rightarrow \mathbb{N}$ is a *sequence flow state function* (where $\mathbb{F}$ is the set of sequence flows and $\mathbb{N}$ is the set of natural numbers) specifying for each sequence flow the current number of tokens marking it; σ_f^0 denotes the state where all sequence flows are unmarked except for the spurious flows, each one marked by one token;
- $\sigma_{doi} : (\mathbb{DO} \times \mathbb{I}) \rightarrow \mathbb{V}$ is a *data object state function* (where $\mathbb{DO}$ is the set of data object names, $\mathbb{I}$ is the set of data object fields, and $\mathbb{V}$ is the set of values) assigning values (possibly null) to data object fields; σ_{doi}^0 denotes the state where all data object attributes are set to null except for those data objects (the so-called *data inputs*) whose initial value is already provided by the modeler;
- $\sigma_{ppa} : (\mathbb{PP} \times \mathbb{A}_{pp}) \rightarrow \mathbb{V}$ is a *physical place attribute state function* assigning values (possibly null) to physical place attributes; σ_{ppa}^0 denotes the state where all place attributes are set to null or the initial value provided by the modeler;
- $\sigma_{ea} : (\mathbb{PP} \times \mathbb{A}_e) \rightarrow \mathbb{V}$ is an *edge attribute state function* assigning values (possibly null) to edge attributes; σ_{ea}^0 denotes the state where all edge attributes are set to null or the initial value provided by the modeler;
- $\sigma_h : \mathbb{P} \rightarrow 2^{\mathbb{P}}$ is a *handshake state function* (where $\mathbb{P}$ is the set of pool names, each one denoting a collaboration participant) that, given a participant, returns the set of participants with which it is bound;
- $\sigma_s : \mathbb{T} \rightarrow \{enab, dis\}$ is a *task state function* (where $\mathbb{T}$ is the set of task names) used to keep track of the active tasks; the status of a task depends on its evolution from enabled (*enab*) to disabled (*dis*) states and vice versa; σ_s^0 denotes the state where all tasks are disabled;
- $\sigma_p : \mathbb{P} \rightarrow \mathbb{PP}$ is the *place state function* mapping each participant to a place; σ_p^0 denotes the state where all places are empty except for those containing the initial positions of participants provided by the modeler;
- $\sigma_t : \mathbb{T} \rightarrow \mathbb{N}$ is a *timer state function* specifying for each task's timer the time units to wait until it expires; σ_t^0 denotes the state where all timers are set to undef;
- $\sigma_m : \mathbb{M} \rightarrow 2^{\mathbb{V}^n}$ is a *message state function* (where $\mathbb{M}$ is the set of message flow names) that assigns to each message flow name m an ordered multiset of values

representing the messages received along m ; σ_m^0 denotes the state where all message multisets are empty.

We define the update of state functions only for σ_f , as the update of the other state functions are similarly defined. The state obtained by updating in σ_f the number of tokens of the sequence flow f to n , written as $\sigma_f \cdot [f \mapsto n]$, is defined as follows: $(\sigma_f \cdot [f \mapsto n])(f')$ returns n if $f' = f$, otherwise it returns $\sigma_f(f')$.

The operational semantics of environment-aware BPMN collaborations is defined employing *labeled transition systems* (LTSs). The LTS of the process behavior is a triple $\langle \mathcal{P}, \mathcal{L}, \rightarrow \rangle$ where: $\mathcal{P}$ is a set of processes configurations; $\mathcal{L}$, ranged over by λ , is a set of labels; and $\rightarrow \subseteq \mathcal{P} \times \mathcal{L} \times \mathcal{P}$ is a transition relation. The labels used by the transition relations $\rightarrow$ are generated by the following grammar:

$$\begin{aligned}
 \eta &::= & (\text{Handshake labels}) \\
 &\rightarrow h \leftarrow & (\text{binding}) \\
 &| \leftarrow h \rightarrow & (\text{unbinding}) \\
 \varphi &::= & (\text{Untimed labels}) \\
 &\eta & (\text{handshake}) \\
 &| \tau & (\text{silent actions}) \\
 \lambda &::= & (\text{Process labels}) \\
 &\varphi & (\text{untimed action}) \\
 &| \checkmark & (\text{timed action})
 \end{aligned}$$

where $\checkmark$ denotes the passing of a unit of time (due, e.g., to movements in the environment), and τ denotes a silent transition (due, e.g., token flow in the processes). As simplification, $\langle P, ENV, \sigma \rangle \xrightarrow{\lambda} \langle P, ENV, \sigma' \rangle$ indicates that $(\langle P, ENV, \sigma \rangle, \lambda, \langle P, ENV, \sigma' \rangle) \in \rightarrow$, and says that the process P and the environment ENV in the state σ can do a transition labeled by λ and evolve to the state σ' . The LTS describing the behavior of collaborations (resp. open-world open-world environments) is a triple $\langle \mathcal{C}, \mathcal{L}', \rightarrow_c \rangle$ (resp. $\langle \mathcal{O}, \mathcal{L}', \rightarrow_o \rangle$) where: $\mathcal{C}$ (resp. $\mathcal{O}$) is a set of collaboration (resp. open-world environment) configurations, $\mathcal{L}'$, ranged over by α , is a set of labels; and $\rightarrow_c \subseteq \mathcal{C} \times \mathcal{L}' \times \mathcal{C}$ (resp. $\rightarrow_o \subseteq \mathcal{O} \times \mathcal{L}' \times \mathcal{O}$) is a transition relation. The labels used by these transition relations are generated by the following grammar:

$$\begin{aligned}
 \alpha &::= & (\text{Collaboration labels}) \\
 &\tau & (\text{silent action}) \\
 &| \checkmark & (\text{timed action})
 \end{aligned}$$

To improve the readability, the following simplifications reduce the notation of operational rules. We omit: (i) the environment ENV from the source configuration of transitions, as it is the same in all rules; (ii) the execution state σ from the source configuration of transitions, except in those rules with more than one transition in the premise; (iii) the process/collaboration structure and the environment from the target configuration of transitions, since they are not affected by transitions; (iv) those state functions from target configurations that are not affected by transitions. Thus, for example, a transition $\langle P, ENV, \sigma_f, \sigma_{doi}, \sigma_{ppa}, \sigma_{ea}, \sigma_s, \sigma_p, \sigma_t, \sigma_m \rangle \xrightarrow{\lambda}$

$\langle P, ENV, \sigma'_f, \sigma'_{doi}, \sigma'_{ppa}, \sigma'_{ea}, \sigma'_s, \sigma'_p, \sigma'_t, \sigma'_m \rangle$ will be written as $P \xrightarrow{\lambda} \langle \sigma'_f \rangle$ when it simply affects the sequence flow state function.

Moreover, to further simplify the definition of the operational rules, in Figure 2 we define *auxiliary functions* that update the state functions of process configurations or performs checks in rules' conditions.

Function $inc(\sigma_f, f) = \sigma_f \cdot [f \mapsto \sigma_f(f) + 1]$, resp. $dec(\sigma_f, f) = \sigma_f \cdot [f \mapsto \sigma_f(f) - 1]$, increments, resp. decrements, by one the number of tokens marking the sequence flow f in the state σ_f . They extend in a natural way to sets F of sequence flows; they are inductively defined as follows: $inc(\sigma_f, \emptyset) = dec(\sigma_f, \emptyset) = \sigma_f$, $inc(\sigma_f, \{f\} \cup F) = inc(inc(\sigma_f, f), F)$, $dec(\sigma_f, \{f\} \cup F) = dec(dec(\sigma_f, f), F)$. Function $dec(\sigma_t, n)$, which decrements the timer value for the timer n , is defined similarly.
Function $enab(\sigma_s, n) = \sigma_s \cdot [n \mapsto enab]$, resp. $dis(\sigma_s, n) = \sigma_s \cdot [n \mapsto dis]$, activates, resp. deactivates, the task n in the state σ_s .
Function $eval(ENV, \sigma_{doi}, \sigma_{ppa}, \sigma_{ea}, \sigma_p, exp) = v$ states that v is the value resulting from the evaluation of the expression exp on the data fields in σ_{doi} , attributes in σ_{ppa} and σ_{ea} , and participants' positions in σ_p . For logical place attributes, the evaluation resorts to the aggregation functions specified in the views of the environment model ENV . For the sake of presentation, we omit the environment and the state functions when they are clear from the context, thus writing $eval(exp)$.
Function $upd(ENV, \sigma_{doi}, \sigma_{ppa}, \sigma_{ea}, \sigma_p, A)$ performs the assignments A and returns the updated data object, physical place attribute and edge attributes state functions. For example, $upd(ENV, \sigma_{doi}, \sigma_{ppa}, \sigma_{ea}, \sigma_p, assign(exp, do.i)) = (\sigma_{doi} \cdot [do.i \mapsto eval(exp)], \sigma_{ppa}, \sigma_{ea})$. Differently, the assignment of a logical place attribute resorts to the disaggregation function specified in the environment model ENV . $upd(ENV, \sigma_{doi}, \sigma_{ppa}, \sigma_{ea}, \sigma_p, assign(exp, lp.a)) = (\sigma_{doi}, \sigma'_{ppa}, \sigma_{ea})$ where σ'_{ppa} is returned by the call of the disaggregation function associated to $lp.a$. Formally, given the view $(\mathbb{L}\mathbb{P}, \mathbb{A}_{lp}, A_{lp})$ in ENV such that $lp \in \mathbb{L}\mathbb{P}$, the disaggregation function corresponding to a is $A_{lp}(a) \downarrow_2 = f_{dag}$ (where $\downarrow_i$ is a projection function that returns the second item of a pair). Thus, the call of disaggregation function is $f_{dag}(lp, eval(exp))$; we recall that aggregation/disaggregation functions are parametric with respect to the physical attribute on which they operate. For the sake of presentation, we omit the environment and the state functions when they are clear from the context, thus writing $upd(A)$.
Function $next(PL, \sigma_p, p, pl)$, where pl can be either a logical place lp or a physical place pp , is defined as follows. If a physical place pp is given as input, i.e. $next(PL, \sigma_p, p, pp)$, it returns the set of places that are next in the paths in the physical layer PL from the current place $\sigma_p(p)$ of the process participant to the destination pp ; the function returns $\emptyset$ when there is no path to the destination. If a logical place lp is given as input, i.e. $next(PL, \sigma_p, p, lp)$, it returns $next(PL, \sigma_p, p, pp)$ where $pp \in \mathcal{P}(lp)$. This means that, when the destination specified by the task is a logical place, the next place is computed, for the sake of simplicity, considering as destination one of the physical places (non-deterministically selected) forming the logical place. Notably, the $next$ function determines the next place to reach by computing the shortest paths to the destination; different definitions can be provided for this function to specify different navigation strategies.
Function $arrived(\sigma_p, p, pl)$, where pl can be either a logical place lp or a physical place pp , returns <i>true</i> if the current position of the participant p is the destination place pl . Formally, $arrived(\sigma_p, p, pp) = (\sigma_p(p) = pp)$, and $arrived(\sigma_p, p, lp) = (\sigma_p(p) \in \mathcal{P}(lp))$.
Function $move(\sigma_p, \sigma_h, p, pp)$ assigns to the participant p , and all participants bound to it, the physical place pp in the state σ_p . Formally, $move(\sigma_p, \sigma_h, p, pp) = \sigma_p \cdot [p \mapsto pp] \cdot [p_1 \mapsto pp] \dots [p_n \mapsto pp]$ with $\sigma_h(p) = \{p_1, \dots, p_n\}$.
Function $set(\sigma_t, n, k) = \sigma_t \cdot [n \mapsto k]$ sets the timer of the task named n to the natural number k .
Function $reset(\sigma_t, n) = \sigma_t \cdot [n \mapsto undef]$ resets the timer of the task named n .
Function $add(\sigma_m, m, v) = \sigma_m \cdot [m \mapsto \sigma_m(m) \uplus \{v\}]$ adds the value v for the message flow m in the state σ_m .
Function $rm(\sigma_m, m, v) = \sigma_m \cdot [m \mapsto \sigma_m(m) \setminus \{v\}]$ removes the value v for message m from the state σ_m .
Function $getD(d)$ can be expressed in different ways. In this paper, duration d is considered as a couple (k_{min}, k_{max}) such that $getD(d)$ returns a uniformly distributed natural number in the interval $[k_{min}, k_{max}]$.
Function $notTimedEl(P)$ returns <i>false</i> when P is a basic task or a movement task, i.e., $notTimedEl(task(f, n, exp, A, d, f')) = notTimedEl(taskM(f, n, exp, A, exp', f')) = false$. For all other elements, this function returns <i>true</i> . Finally $notTimedEl(P_1 \mid P_2) = notTimedEl(P_1) \wedge notTimedEl(P_2)$.
Function $bind(\sigma_h, p_1, p_2)$ binds together the participants p_1 and p_2 in σ_h . Formally, $bind(\sigma_h, p_1, p_2) = \sigma'_h$ with $\sigma'_h(p) = \sigma_h(p)$ if $p \notin \{p_1, p_2\}$, $\sigma'_h(p_1) = \sigma_h(p_1) \cup \{p_2\}$, and $\sigma'_h(p_2) = \sigma_h(p_2) \cup \{p_1\}$. Notation $\sigma \cdot \sigma'_h$ denotes the update of state σ by replacing the enclosed σ_h by σ'_h .
Function $unbind(\sigma_h, p_1, p_2)$ unbinds the participants p_1 and p_2 in σ_h . Formally, $unbind(\sigma_h, p_1, p_2) = \sigma'_h$ with $\sigma'_h(p) = \sigma_h(p)$ if $p \notin \{p_1, p_2\}$, $\sigma'_h(p_1) = \sigma_h(p_1) \setminus \{p_2\}$, and $\sigma'_h(p_2) = \sigma_h(p_2) \setminus \{p_1\}$.
Function $evolve(\sigma)$ updates the environmental attributes in the state σ independently from the activities performed by the collaboration participants. The definition of this function is application dependent and must be provided by the modeller when the collaboration is executed in an open-world environment. By default, $evolve(\sigma) = \sigma$, meaning that the environment does not evolve independently if the modeler does not specify how.

Fig. 2: Auxiliary functions

The operational rules defining the transition relations of the environment-aware BPMN semantics are given by the inference rules in Figures 3, 4, and 5.

We now briefly comment on the semantic rules, starting from those dealing with events and gateways in Figures 3. Rule *P-Start* starts the execution of a process when it has been activated. To denote the enabled status of start events we have included in their syntactical definition an incoming (spurious) sequence flow, named enabling sequence flow. Thus, the process is activated when the enabling sequence flow of a start event is marked. The effect of the rule is to increment the number of tokens in the sequence flow outgoing from the start event and to decrease the marking of the enabling sequence flow. Rule *P-End* instead is enabled when there is at least one token in the incoming sequence flow of the end event, which is then consumed. Rule *P-Rcv* is enabled when there is at least a token in the incoming flow and when there is a value in the message queue. As a result, this function increments the number of tokens in the outgoing sequence flow, updates the data object fields with the received value, and removes it from the queue. Rule *P-Snd* is enabled when there is at least a token in its incoming flow; as a result, the value resulting from the evaluation of the message expression exp is added to the queue of the message flow m , and the number of tokens in the outgoing sequence flow is incremented. Rule *P-AndSplit* is applied when there is at least one token in the incoming edge of an AND split gateway; as a result of its application, the rule decrements the number of tokens in the incoming sequence flow, and increments the tokens in each outgoing sequence flow. Rule *P-XorSplit* is applied when a token is available in the incoming sequence flow of a XOR split gateway and a conditional expression of one of its outgoing sequence flows is evaluated to true; the rule decrements the token in the incoming sequence flow and increments the token in the selected outgoing sequence flow. Notably, if more edges have their guards satisfied, one of them is non-deterministically chosen. Rule *P-XorJoin* is activated every time there is a token in one of the incoming sequence flows, which is then moved to the outgoing sequence flow. Rule *P-EventB* is activated when there is a token in the incoming flow and a value is present for at least one of the connected messages. The effect of the rule is similar to that of rule *P-Rcv*.

Let us consider the rules in Figure 4, dealing with basic tasks, movement tasks, binding and unbinding tasks. Rules *P-Task_{Act}* and *P-TaskM_{Act}* both activate tasks and movement tasks when they are disabled, there is a token in the incoming sequence flow, and the guard exp_g is satisfied. Then, they remove the token in the incoming sequence flow, enable the tasks, and start the timer of the task (not for the movement task). Rule *P-Task_{Elps1}* decrements the value of the task's timer when it is enabled and the timeout is not expired. Rules *P-Task_{Elps2}* and *P-TaskM_{Elps2}* enables tasks and movement tasks to evolve with temporal transitions when they are disabled. Rule *P-Task_C* deals with the completion of a basic task. If the task is enabled and the timeout is expired, the rule increments the token in the outgoing sequence flow, performs the assignments, disables the task, and resets the timer. Rule *P-TaskM_{Elps1}* performs a movement to the next place of the physical layer of the environment when the task is enabled and the next physical place is reachable. Rule *P-TaskM_{Elps3}* enables movement tasks to evolve with temporal transitions when it is enabled, the destination is not reachable, and the current position is not the destination. Rule *P-TaskM_C* deals with the completion of the movement task; if the task is enabled and the destination has been reached, the rule increments the outgoing sequence flow, performs the assignments, and disables the task. Rules *P-TaskB*

$\text{start}(f, f') \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f') \rangle$	$\sigma_f(f) > 0$	(P-Start)
$\text{end}(f) \xrightarrow{\tau} \langle \text{dec}(\sigma_f, f) \rangle$	$\sigma_f(f) > 0$	(P-End)
$\text{rcv}(f, m, \text{do.i}, f') \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f'), \text{upd}(\text{assign}(v, \text{do.i})), \text{rm}(\sigma_m, m, v) \rangle$	$\sigma_f(f) > 0 \wedge v \in \sigma_m(m)$	(P-Rcv)
$\text{snd}(f, m, \text{exp}, f') \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f'), \text{add}(\sigma_m, m, v) \rangle$	$\sigma_f(f) > 0 \wedge \text{eval}(\text{exp}) = v$	(P-Snd)
$\text{andSplit}(f, F) \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), F) \rangle$	$\sigma_f(f) > 0$	(P-AndSplit)
$\text{xorSplit}(f, (\text{exp}', f')(\text{exp}'', f'')^*) \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f') \rangle$	$\sigma_f(f) > 0 \wedge \text{eval}(\text{exp}') = \text{true}$	(P-XorSplit)
$\text{andJoin}(F, f) \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, F), f) \rangle$	$\forall f' \in F . \sigma_f(f') > 0$	(P-AndJoin)
$\text{xorJoin}(\{f\} \cup F, f') \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f') \rangle$	$\sigma_f(f) > 0$	(P-XorJoin)
$\text{eventBased}(f, (m, \text{do.i}, f')(m', \text{do'.i}', f'')^*) \xrightarrow{\tau} \langle \text{inc}(\text{dec}(\sigma_f, f), f'), \text{upd}(\text{assign}(v, \text{do.i})), \text{rm}(\sigma_m, m, v) \rangle$	$\sigma_f(f) > 0 \wedge v \in \sigma_m(m)$	(P-EventB)

Fig. 3: Environment-aware BPMN process semantics: events and gateways.

and *P-TaskU* deals with binding and unbinding tasks; if there is a token in the incoming sequence flow of those tasks, the effect of the rules is to increment the number of tokens in the outgoing sequence flow and to decrease the token in the incoming sequence flow, while producing a transition label representing the will of the participant to bind/unbind with the other participant connected by the handshake flow h .

Finally, we comment on the operational rules in Figure 5, dealing with the interleaving of process and collaboration elements. Rule *P-Elps* enables not timed process elements to evolve the process configuration with temporal transitions. Rule *P-Int₁* enables single process elements to evolve the process configuration with silent transitions, instead rule *P-Int₂* enables process elements to evolve the process configuration with temporal transitions $\checkmark$ if no silent actions φ may occur (see maximal progress assumption of timed process calculi [3]). Regarding collaboration elements, rule *C-Prop* propagates the transition from process to collaboration configuration, while *C-Int₁* and *C-Int₂* behave as *P-Int₁* and *P-Int₂*, respectively. Notably, rules *P-Int₁* and *C-Int₁* allow single process elements to evolve the configuration, while *P-Int₂* and *C-Int₂* imposes a synchronous triggering of rules producing a temporal transition. Rule *C-Bind* (resp. *C-Unbind*) binds (resp. unbinding) two participants if they are in the same physical place. Rule *C-Open₁* allows the environmental attributes of the configuration to evolve in an open-world fashion, i.e. independently from the collaboration execution. Rule *C-Open₂*, instead, does not allow the environment to evolve independently when there is no elapse of time.

$\text{task}(f, n, \text{exp}_g, A, d, f') \xrightarrow{\tau} \langle \text{dec}(\sigma_f, f), \text{enab}(\sigma_s, n), \text{set}(\sigma_t, n, \text{getD}(d)) \rangle$	$\sigma_f(f) > 0 \wedge \sigma_s(n) = \text{dis} \wedge \text{eval}(\text{exp}_g) = \text{true}$	$(P\text{-Task}_{\text{Act}})$
$\text{task}(f, n, \text{exp}_g, A, d, f') \xrightarrow{\checkmark} \langle \text{dec}(\sigma_t, n) \rangle$	$\sigma_s(n) = \text{enab} \wedge \sigma_t(n) > 0$	$(P\text{-Task}_{\text{Elps1}})$
$\text{task}(f, n, \text{exp}_g, A, d, f') \xrightarrow{\checkmark} \langle \rangle$	$\sigma_s(n) = \text{dis}$	$(P\text{-Task}_{\text{Elps2}})$
$\text{task}(f, n, \text{exp}_g, A, d, f') \xrightarrow{\tau} \langle \text{inc}(\sigma_f, f'), \text{upd}(A), \text{dis}(\sigma_s, n), \text{reset}(\sigma_t, n) \rangle$	$\sigma_s(n) = \text{enab} \wedge \sigma_t(n) = 0$	$(P\text{-Task}_C)$
$\text{taskM}(f, n, \text{exp}_g, A, \text{exp}_d, f') \xrightarrow{\tau} \langle \text{dec}(\sigma_f, f), \text{enab}(\sigma_s, n) \rangle$	$\sigma_f(f) > 0 \wedge \sigma_s(n) = \text{dis} \wedge \text{eval}(\text{exp}_g) = \text{true}$	$(P\text{-TaskM}_{\text{Act}})$
$\text{taskM}(f, n, \text{exp}_g, A, \text{exp}_d, f') \xrightarrow{\checkmark} \langle \text{move}(\sigma_p, \sigma_h, p, pp) \rangle$	$\sigma_s(n) = \text{enab} \wedge pp \in \text{next}(PL, \sigma_p, p, \text{eval}(\text{exp}_d))$	$(P\text{-TaskM}_{\text{Elps1}})$
$\text{taskM}(f, n, \text{exp}_g, A, \text{exp}_d, f') \xrightarrow{\checkmark} \langle \rangle$	$\sigma_s(n) = \text{dis}$	$(P\text{-TaskM}_{\text{Elps2}})$
$\text{taskM}(f, n, \text{exp}_g, A, \text{exp}_d, f') \xrightarrow{\checkmark} \langle \rangle$	$\sigma_s(n) = \text{enab} \wedge \text{next}(PL, \sigma_p, p, \text{eval}(\text{exp}_d)) = \emptyset \wedge \neg \text{arrived}(\sigma_p, p, \text{eval}(\text{exp}_d))$	$(P\text{-TaskM}_{\text{Elps3}})$
$\text{taskM}(f, n, \text{exp}_g, A, \text{exp}_d, f') \xrightarrow{\tau} \langle \text{inc}(\sigma_f, f'), \text{upd}(A), \text{dis}(\sigma_s, n) \rangle$	$\sigma_s(n) = \text{enab} \wedge \text{arrived}(\sigma_p, p, \text{eval}(\text{exp}_d))$	$(P\text{-TaskM}_C)$
$\text{taskB}(f, h, f) \xrightarrow{\rightarrow h \leftarrow} \langle \text{inc}(\text{dec}(\sigma_f, f), f') \rangle$	$\sigma_f(f) > 0$	$(P\text{-TaskB})$
$\text{taskU}(f, h, f) \xrightarrow{\leftarrow h \rightarrow} \langle \text{inc}(\text{dec}(\sigma_f, f), f') \rangle$	$\sigma_f(f) > 0$	$(P\text{-TaskU})$

Fig. 4: Environment-aware BPMN process semantics: tasks and movement tasks.

3 Case Study Formalization

To assess our formalization, we propose a case study concerning an emergency response collaboration process, which occurs within a hospital and its surroundings. The environment model depicted in Figure 6 overall comprises 32 physical places, 6 logical places and 3 views. The *Departments* view includes the *Emergency Department* and *Radiology Department* logical places. The *Available Rooms* view includes the *Available Emergency Rooms* and *Available Radiology Rooms* logical places. The *Roads* view comprises the *SP17* and *SP256* logical places. The collaboration, see Figure 6, is performed by four participants: *Injured Patient*, *Ambulance*, *Emergency Nurse* and *Emergency Doctor*.

The emergency response collaboration depicted in Figure 7 starts when the *Injured Patient* is involved in a car accident and calls an ambulance, providing its position. Upon receiving the call, the *Ambulance* moves to the patient's position. Once arrived, the am-

$P \xrightarrow{\checkmark} \langle \rangle \quad \text{notTimedEl}(P) = \text{true} \quad (P\text{-Elps})$	
$\frac{P_1 \xrightarrow{\emptyset} \langle \sigma' \rangle}{P_1 \mid P_2 \xrightarrow{\emptyset} \langle \sigma' \rangle} \quad (P\text{-Int}_1)$	$\frac{P_1 \xrightarrow{\checkmark} \langle \sigma' \rangle \quad \langle P_2, \sigma' \rangle \xrightarrow{\checkmark} \langle \sigma'' \rangle \quad P_1 \mid P_2 \xrightarrow{\emptyset} \langle \sigma' \rangle}{P_1 \mid P_2 \xrightarrow{\checkmark} \langle \sigma'' \rangle} \quad (P\text{-Int}_2)$
$\frac{\langle P, ENV, p, \sigma \rangle \xrightarrow{\lambda} \langle \sigma' \rangle}{\langle \text{pool}(p, P), ENV, \sigma \rangle \xrightarrow{\lambda} \langle \sigma' \rangle} \quad \lambda \neq \eta \quad (C\text{-Prop})$	
$\frac{P_1 \xrightarrow{h\leftarrow} \langle \sigma' \rangle \quad \langle P_2, \sigma' \rangle \xrightarrow{h\leftarrow} \langle \sigma'' \rangle \quad \sigma_p(p_1) = \sigma_p(p_2)}{\text{pool}(p_1, P_1) \parallel \text{pool}(p_2, P_2) \xrightarrow{\tau} \langle \sigma'' \cdot \text{bind}(\sigma_h, p_1, p_2) \rangle} \quad (C\text{-Bind})$	
$\frac{P_1 \xrightarrow{h\rightarrow} \langle \sigma' \rangle \quad \langle P_2, \sigma' \rangle \xrightarrow{h\rightarrow} \langle \sigma'' \rangle \quad \sigma_p(p_1) = \sigma_p(p_2)}{\text{pool}(p_1, P_1) \parallel \text{pool}(p_2, P_2) \xrightarrow{\tau} \langle \sigma'' \cdot \text{unbind}(\sigma_h, p_1, p_2) \rangle} \quad (C\text{-Unbind})$	
$\frac{C_1 \xrightarrow{\tau} \langle \sigma' \rangle}{C_1 \parallel C_2 \xrightarrow{\tau} \langle \sigma' \rangle} \quad (C\text{-Int}_1)$	$\frac{C_1 \xrightarrow{\checkmark} \langle \sigma' \rangle \quad \langle C_2, \sigma' \rangle \xrightarrow{\checkmark} \langle \sigma'' \rangle \quad C_1 \parallel C_2 \xrightarrow{\tau} \langle \sigma' \rangle}{C_1 \parallel C_2 \xrightarrow{\checkmark} \langle \sigma'' \rangle} \quad (C\text{-Int}_2)$
$\frac{\langle C, ENV, \sigma \rangle \xrightarrow{\checkmark} \langle \sigma' \rangle}{[C, ENV, \sigma] \xrightarrow{\checkmark} \langle \text{evolve}(\sigma') \rangle} \quad (C\text{-Open}_1)$	$\frac{\langle C, ENV, \sigma \rangle \xrightarrow{\tau} \langle \sigma' \rangle}{[C, ENV, \sigma] \xrightarrow{\tau} \langle \sigma' \rangle} \quad (C\text{-Open}_2)$

Fig. 5: Environment-aware BPMN process and collaboration semantics.

balance picks up the patient with the bind tasks *Get into ambulance* and *Load patient into ambulance* and transports him to the hospital. Once at the hospital, the ambulance leaves the patient to the hospital with an unbind task. The patient is then received by the *Emergency Nurse*, who escorts him to an available emergency room and send his information to the *Emergency Doctor*. Upon arrival, the *available* attribute of the respective physical place is updated with the disaggregation function *occupy*. The Emergency doctor, after reviewing the patient's information, examines the patient upon his arrival with the nurse. Following the examination, the doctor provides a diagnosis and instructs the nurse on which room the patient should be taken to for further tests. The nurse then escorts the patient to the designated room and leaves him for the necessary analyses.

In Figure 8, we provide the textual representation of the presented case study. This exemplifies the correspondence between the graphical representation of the environment and the proposed BNF syntax. Note that the edges are undirected in the example, so they are represented by two directed edges with the same edge id, having the same attribute values.

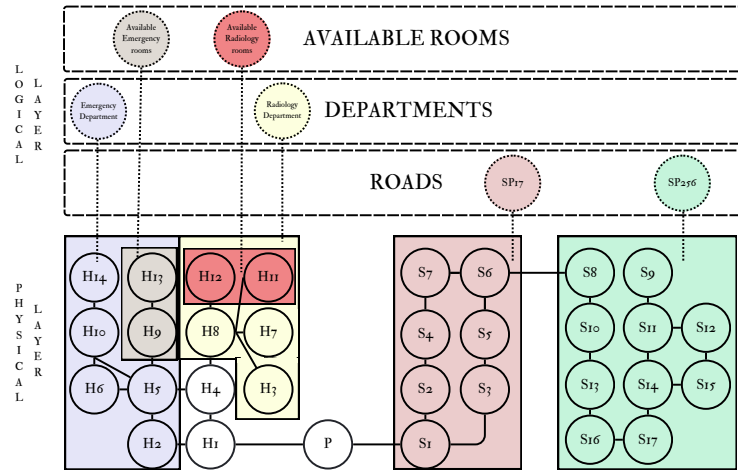


Fig. 6: Case Study Environment Representation

References

1. Corradini, F., Piccioni, J., Re, B., Rossi, L., Tiezzi, F.: On the Interplay Between BPMN Collaborations and the Physical Environment. In: International Conference of Business Process Management, BPM2024. p. 93–110. Springer-Verlag (2024)
2. OMG: Business process model and notation. (BPMN V2.0) (2011)
3. Roeber, W.P.d., Hooman, J.: Design and verification in real-time distributed computing: an introduction to compositional methods. In: International Symposium on Protocol Specification, Testing and Verification. p. 37–56. North-Holland Publishing (1989)

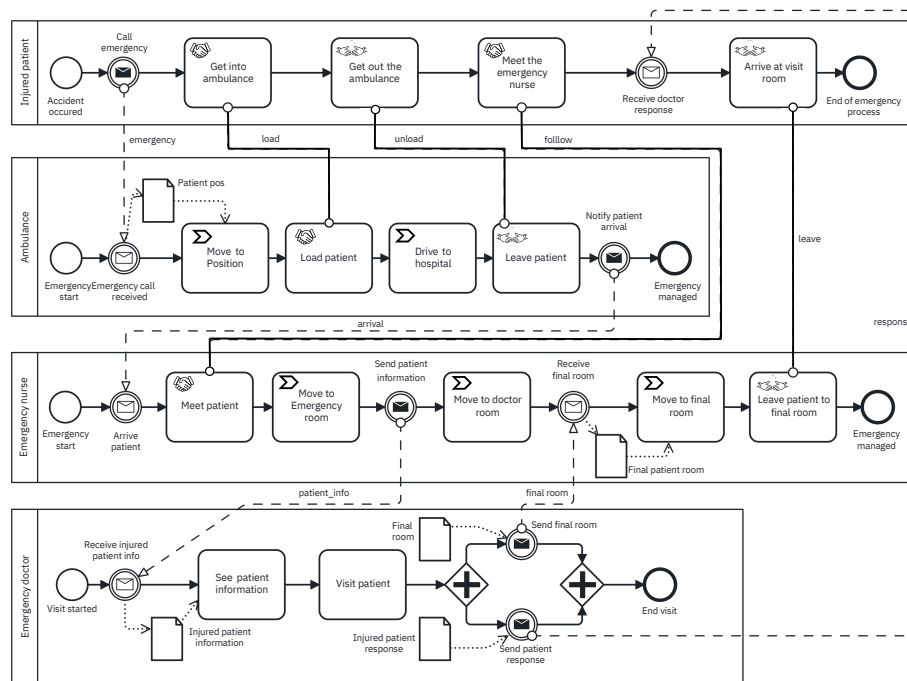


Fig. 7: Emergency response collaboration process

$C_{er} = \text{pool}(P_{patient}, P_{patient}) \parallel \text{pool}(P_{ambulance}, P_{ambulance}) \parallel \text{pool}(P_{nurse}, P_{nurse}) \parallel \text{pool}(P_{doctor}, P_{doctor})$
$P_{patient} = \text{start}(f'_1, f_1) \mid \text{snd}(f_1, m_{emergency}, v_{s14}, f_2) \mid \text{taskB}(f_2, h_{load}, f_3) \mid \text{taskU}(f_3, h_{unload}, f_4) \mid$ $\text{taskB}(f_4, h_{follow}, f_5) \mid \text{rcv}(f_5, m_{response}, do_{response} \cdot i_{msg}, f_6) \mid \text{taskU}(f_6, h_{leave}, f_7) \mid \text{end}(f_7)$
$P_{ambulance} = \text{start}(f'_8, f_8) \mid \text{rcv}(f_8, m_{emergency}, do_{PatientPos} \cdot i_{position}, f_9) \mid \text{taskM}(f_9, n_{MovetoPosition}, do_{PatientPos} \cdot i_{position}, f_{10}) \mid$ $\text{taskB}(f_{10}, h_{load}, f_{11}) \mid \text{taskM}(f_{11}, n_{DrivetoHospital}, pp_{H1}, f_{12}) \mid \text{taskU}(f_{12}, h_{leave}, f_{13}) \mid$ $\text{snd}(f_{13}, m_{arrival}, v_{patientarrival}, f_{14}) \text{end}(f_{14})$
$P_{nurse} = \text{start}(f'_{15}, f_{15}) \mid \text{rcv}(f_{15}, m_{arrival}, do_{Info} \cdot i_{info}, f_{16}) \mid \text{taskB}(f_{16}, h_{follow}, f_{17}) \mid$ $\text{taskM}(f_{17}, n_{MovetoEmergencyRoom}, (lp_{availableStudyRooms} \cdot a_{seats} > 0), A_1, lp_{AvailableEmergencyrooms}, f_{18}) \mid$ $\text{snd}(f_{18}, m_{patientinfo}, v_{patientinfo}, f_{19}) \mid \text{taskM}(f_{19}, n_{MovetoDoctorRoom}, pp_{H4}, f_{20}) \mid$ $\text{rcv}(f_{20}, m_{finalroom}, do_{FinalRoom} \cdot i_{position}, f_{21}) \mid \text{taskM}(f_{21}, n_{MovetoFinalroom}, do_{FinalRoom} \cdot i_{position}, f_{22}) \mid$ $\text{taskU}(f_{22}, h_{leave}, f_{23}) \mid \text{end}(f_{23})$ $A_1 = \text{assign}((myplace.a_{freeSeats} - 1), myplace.a_{freeSeats})$
$P_{doctor} = \text{start}(f'_{24}, f_{24}) \mid \text{rcv}(f_{24}, m_{patientinfo}, do_{InjuredPatientInformation} \cdot i, f_{25}) \mid \text{task}(f_{25}, n_{SeePatientInformation}, d_1, f_{26}) \mid$ $\text{task}(f_{26}, n_{VisitPatient}, l, f_{27}) \mid \text{andSplit}(f_{27}, (f_{28}, f_{29})) \mid \text{snd}(f_{28}, m_{response}, v_{patientresponse}, f_{30}) \mid$ $\text{snd}(f_{29}, m_{finalroom}, v_{pp_{H8}}, f_{31}) \mid \text{andSplit}(f_{30}, (f_{31}, f_{32})) \mid \text{end}(f_{32})$
$ENV_h = (PL_h, LL_h)$ $PL_h = (\{pp_{H1}, \dots, pp_{H14}, pp_P, pp_{S1}, \dots, pp_{S17}\}, \{e_1, \dots, e_{35}\}, \mathbb{A}_{pp} : \{a_{zone}, a_{purpose}, a_{freeSeats}\}, \mathbb{A}_e : \{a_{status}\},$ $E = \{(\{pp_{H1}, e_1, pp_{H2}\}, \{pp_{H2}, e_2, pp_{H1}\}), \dots, (\{pp_{S17}, e_{35}, pp_{S16}\}, \{pp_{S16}, e_{20}, pp_{S17}\}), \}$ $LL_h = (V_{Departments}, V_{AvailableRooms}, V_{Roads}, LP_h)$ $LP_h : [lp_{EmergencyDepartment} \mapsto a_{zone} == v_{hospital} \text{ and } a_{purpose} == v_{emergency},$ $lp_{Radiology} \mapsto a_{zone} == v_{hospital} \text{ and } a_{purpose} == v_{radiology},$ $lp_{AvailableRadiologyRooms} \mapsto a_{zone} == v_{hospital} \text{ and } a_{purpose} == v_{radiology} \text{ and } a_{freeSeats} > 0,$ $lp_{AvailableEmergencyRooms} \mapsto a_{zone} == v_{hospital} \text{ and } a_{purpose} == v_{emergency} \text{ and } a_{freeSeats} > 0,$ $lp_{SP276} \mapsto a_{zone} == v_{street} \text{ and } a_{purpose} == v_{SP176},$ $lp_{SP17} \mapsto a_{zone} == v_{street} \text{ and } a_{purpose} == v_{SP17}]$ $V_{Departments} : (\{lp_{EmergencyDepartment}, lp_{RadiologyDepartment}\}, \emptyset, \emptyset)$ $V_{AvailableRooms} : (\{lp_{AvailableEmergencyRooms}, lp_{AvailableRadiologyRooms}\}, \{a_{seats}\}, [a_{seats} \mapsto (sum(a_{freeSeats}), occupy(a_{freeSeats}))])$ $V_{Roads} : (\{lp_{SP17}, lp_{SP276}\}, \emptyset, \emptyset)$ $sum(a)(this) : \quad \quad \quad occupy(a)(this, n) :$ $res = 0; \quad \quad \quad \text{forEach } pp \text{ in } \mathcal{P}(this)$ $\text{forEach } pp \text{ in } \mathcal{P}(this) \quad \text{if } n > pp.a \text{ then } n- = pp.a; \quad pp.a = 0;$ $res+ = pp.a; \quad \quad \quad \text{else } pp.a- = n;$ $\text{return } res; \quad \quad \quad \text{return};$

Fig. 8: Spatial Textual representation of the university example